

Assignment Report

student-ml-api

Flask ML API with CI/CD Pipeline

Repository: github.com/Hhumna/student-ml-api

Current Version: 1.2.1

Table of Contents

- Table of Contents 2
- 1. Project Overview 3
 - 1.1 Endpoints..... 3
 - 1.2 Tech Stack..... 3
- 2. Application Code (app.py) 3
- 3. Automated Tests (tests/test_app.py) 3
- 4. Dockerization (Dockerfile)..... 4
- 5. CI/CD Pipeline (GitHub Actions) 4
 - 5.1 CI Workflow (ci.yml) 4
 - 5.2 Release Workflow (release.yml)..... 4
- 6. Branch Protection & Development Workflow 4
 - 6.1 Development Flow 5
- 7. Releases & Versioning (Traceability Table) 5
- 8. Docker Build Cache Analysis..... 5
- 9. Rollback Demonstration 6
- 10. Failure Analysis (Intentional & Accidental) 6
 - 10.1 Failure 1 — Failed pytest in CI 6
 - 10.2 Failure 2 — Wrong container port mapping 6
- 11. Files Created / Modified 6

1. Project Overview

A Flask-based machine learning inference REST API, named student-ml-api, was built and integrated with a complete CI/CD pipeline using GitHub Actions and the GitHub Container Registry (GHCR).

1.1 Endpoints

Endpoint	Description
GET /health	Returns application health status and metadata (status, application name, version, model version).
POST /predict	Accepts a JSON body with a numeric "value" field and returns a prediction (value × 2), with full input validation and error handling.

1.2 Tech Stack

- Language: Python 3.11
- Framework: Flask
- Testing: pytest
- Container: Docker (python:3.11-slim base image)
- CI/CD: GitHub Actions
- Registry: GitHub Container Registry (GHCR)

2. Application Code (app.py)

- Created app.py with the core Flask application setup.
- Implemented GET /health returning JSON with status, application name, application_version (read dynamically from a VERSION file), and model_version ("model-1").
- Implemented POST /predict with input validation (missing "value" key → 400 error), type validation (non-numeric values such as strings or booleans → 400 error), and prediction logic of value × 2.
- Added a VERSION file, read at startup, so the running container always reports its own version.

3. Automated Tests (tests/test_app.py)

Six pytest test cases were written, covering all major code paths. All six tests pass on every CI run.

#	Test	Purpose
1	test_health_endpoint	Checks 200 status and all JSON fields in the response.
2	test_predict_endpoint_success	Valid input (value=5) returns prediction=10.
3	test_predict_endpoint_missing_value	Missing key returns 400 with the correct error message.
4	test_predict_endpoint_empty_json	Empty body returns 400.
5	test_predict_endpoint_invalid_type_string	String value returns 400.
6	test_predict_endpoint_invalid_type_boolean	Boolean value returns 400.

4. Dockerization (Dockerfile)

- Created a multi-layered Dockerfile based on python:3.11-slim.
- Used optimized layer ordering (COPY requirements.txt → RUN pip install → COPY . .) to maximize Docker cache efficiency, so pip install only re-runs when dependencies actually change.
- Added OCI-standard image labels via build-time ARGS: org.opencontainers.image.version, .revision, .source, and .created.
- Application listens on port 5000 (EXPOSE 5000).
- Added a .dockerignore file to exclude unnecessary files from the build context.

5. CI/CD Pipeline (GitHub Actions)

Two separate workflows were created with distinct responsibilities.

5.1 CI Workflow (ci.yml)

Trigger: Push to any feature/** branch, or a pull request targeting main

- Checkout code
- Set up Python 3.11 (with pip caching)
- Install dependencies (pip install -r requirements.txt)
- Run all pytest tests
- Build the Docker image (validates the Dockerfile; does not push)

This workflow acts as a quality gate: it validates that the code is correct and the container builds before anything reaches main.

5.2 Release Workflow (release.yml)

Trigger: Push of a version tag matching v*.*.* (e.g., v1.0.0)

- Checkout code
- Set up Python 3.11 (with pip caching)
- Install dependencies
- Run all pytest tests (final safety check before publish)
- Extract the version number from the Git tag
- Log in to GitHub Container Registry (GHCR)
- Build the Docker image with build args: APP_VERSION, GIT_COMMIT (full SHA), BUILD_DATE (UTC ISO timestamp); tag with the version tag, "latest", and the commit SHA
- Push all three tags to GHCR

Design rationale: separating CI (validate) from Release (publish) ensures only reviewed, merged, and intentionally versioned code ever reaches the container registry, keeping it a trustworthy source of shippable artifacts.

6. Branch Protection & Development Workflow

A GitHub branch protection ruleset was configured on main that:

- Blocks all direct pushes to main
- Requires all changes to go through a pull request
- Requires the CI workflow ("test-and-build" status check) to pass before a PR can be merged

6.1 Development Flow

feature branch -> PR -> CI runs -> code review -> merge to main -> push version tag -> Release workflow -> image published to GHCR

Branch protection was verified by attempting a direct push to main, which was correctly rejected by GitHub with:

"GH013: Repository rule violations found."

Root cause and fix: the ruleset initially targeted the wrong default branch (feature/prediction-api instead of main). After correcting the default branch and the ruleset target, branch protection was confirmed fully operational.

7. Releases & Versioning (Traceability Table)

Three versions were released over the course of the assignment.

Version	PR	Merge Commit	Git Tag	GHCR Image Tag
1.0.0	#1	e2bcd91	v1.0.0	ghcr.io/hhumna/student-ml-api:1.0.0
1.1.0	#2	b73b3b6c	v1.1.0	ghcr.io/hhumna/student-ml-api:1.1.0
1.2.1	#5	a8a571e	v1.2.1	ghcr.io/hhumna/student-ml-api:1.2.1

Note on v1.2.0: it was released with a stale VERSION file — the registry tag said 1.2.0 but /health reported 1.1.0. This was caught, fixed in PR #5, and republished as v1.2.1. It demonstrated the importance of verifying the runtime behavior of a freshly-pulled image, not just the registry tag.

8. Docker Build Cache Analysis

An experiment was conducted comparing three Docker builds: a baseline (fresh) build, a build after changing only app.py, and a build after changing only requirements.txt.

Layer	Baseline	app.py changed	requirements.txt changed
FROM python:3.11-slim	Ran	Cached	Cached
ARG / LABEL instructions	Ran	Cached	Cached
WORKDIR /app	Ran	Cached	Cached
COPY requirements.txt .	Ran	Cached	Re-ran
RUN pip install	Ran	Cached	Re-ran
COPY . .	Ran	Re-ran	Re-ran
EXPOSE / CMD	Ran	Re-ran	Re-ran

Key finding: changing only app.py kept pip install fully cached, while changing requirements.txt forced a full dependency reinstall. This validated the layer-ordering strategy used in the Dockerfile.

9. Rollback Demonstration

- Demonstrated registry-based rollback after v1.1.0 was published.
- Ran the v1.0.0 image directly from GHCR:

```
docker run ghcr.io/hhumna/student-ml-api:1.0.0
```

- Confirmed via /health that the old response schema was restored — no code changes and no rebuild were required, and the rollback completed in seconds.

Advantage over source-based rollback: pulling a pre-built image avoids dependency resolution, interpreter version mismatches, and network-dependent reinstalls — all critical during a live incident.

10. Failure Analysis (Intentional & Accidental)

10.1 Failure 1 — Failed pytest in CI

Symptom: CI run failed with a red X on the feature/prediction-api branch (commit fa0d380).

Root Cause: A test assertion was deliberately changed to expect `data["status"] == "wrong"`, to validate that CI catches test failures.

Evidence: "AssertionError: assert 'healthy' == 'wrong'" after 9 seconds.

Fix: The assertion was reverted in commit 8281357; the next CI run passed with 6/6 tests green.

10.2 Failure 2 — Wrong container port mapping

Symptom: `curl http://localhost:5000/health` returned "Connection refused".

Root Cause: The container was started with `-p 6000:5000` (host:container), so the app was reachable on host port 6000, not 5000.

Evidence: `curl localhost:5000` failed; `curl localhost:6000` succeeded with the correct JSON response.

Fix: Corrected to `-p 5000:5000` in the docker run command.

11. Files Created / Modified

File	Purpose
app.py	Main Flask application
requirements.txt	Python dependencies (Flask, pytest)
VERSION	Tracks current app version (1.2.1)
Dockerfile	Container build instructions
.dockerignore	Excludes unnecessary files from the image
.gitignore	Excludes <code>__pycache__</code> / <code>venv</code> from git
tests/test_app.py	6 pytest test cases
.github/workflows/ci.yml	CI workflow (PR validation)
.github/workflows/release.yml	Release workflow (publish to GHCR)
README.md	Full documentation of all of the above

End of Report